
RESAE: RESIDUALIZED SPARSE AUTOENCODERS FOR MULTI-LAYER TRANSFORMER INTERVENTIONS

Prathyush Poduval^{*,a}, Calvin Yeung^a, Neel Desai^b, Mohsen Imani^a

^aDepartment of Computer Science
University of California, Irvine
Irvine, CA 92697

^bGeorgia Institute of Technology
Atlanta, GA 30332

*Corresponding author: ppoduval@uci.edu

May 28, 2026

ABSTRACT

Sparse autoencoders are usually trained one layer at a time, even though transformer residual stream activations are strongly coupled across depth. This creates a practical problem for multi-layer interventions: different layerwise dictionaries can spend capacity representing the same carried-forward information, and replacing several layers at once can produce interactions that are not predicted by single-layer behavior. We introduce Residualized Sparse Autoencoders (ReSAEs), which fit an affine map between selected layers and train each later-layer SAE on the unexplained residual rather than on the full activation. Reconstructions are mapped back into the original activation space through the fitted affine chain, so ReSAEs can be evaluated with the same intervention protocols as ordinary SAEs. On Pythia-1.4B and Gemma-2-9B, residualization reduces decoder redundancy and improves sparse probing and targeted perturbation in most tested settings. Despite reconstructing less of the raw activation variance, ReSAEs recover more transformer cross entropy under multi-layer replacement. This gain is clearest under teacher-forcing and at sufficient sparsity online, indicating that ReSAEs preserve the components of the activation most relevant to the model’s downstream computation. These results suggest that removing linearly predictable cross-layer structure is a useful default for multi-layer SAE interventions.

1 Introduction

As large language models (LLMs) scale in capability and deployment, understanding their internal representation has become a major point of research. A range of poorly understood behaviors like hallucinations Ji et al. (2023), jailbreaks Zou et al. (2023), emergent misalignment Betley et al. (2025), and sycophancy Sharma et al. (2023) remain difficult to predict or prevent without mechanistic understanding of how information is structured inside these models. The goal of interpretability is to identify what representations the model builds, how they evolve across layers, and how interventions on them can be made reliable.

Sparse autoencoders (SAEs) have emerged as a central tool for this goal. An SAE decomposes a transformer’s internal activations into sparse combinations of directions drawn from a learned dictionary. These individual active directions have been shown to correspond to interpretable, monosemantic concepts in many cases Bricken et al. (2023); Cunningham et al. (2023). Several variants improve upon the basic formulation: BatchTopK Gao et al. (2025) and JumpReLU Rajamanoharan et al. (2024) improve sparsity control and training stability, p-Anneal Karvonen et al. (2024b) achieves better decomposition at lower sparsity, and Matryoshka SAEs Bussmann et al. (2025) learn monosemantic and hierarchical features. Large-scale public releases like GemmaScope Lieberum et al. (2024), LlamaScope He et al. (2024), and SAELens Bloom et al. (2024) have trained SAEs across many layers of frontier-scale models, establishing sparse feature dictionaries as a practical vocabulary for inspecting model internals.

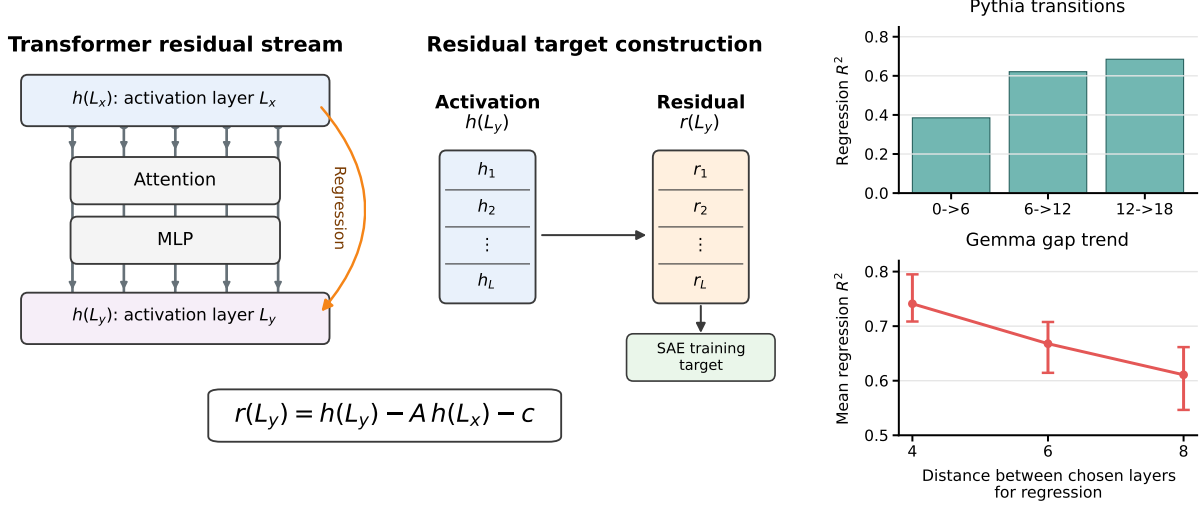


Figure 1: Residualization overview and cross-layer regression R^2 . **Left**: selected transformer activations are connected through the residual stream, with an affine regression mapping an earlier activation $h(L_x)$ to a later activation $h(L_y)$. **Middle**: ReSAEs train the later-layer SAE on the residual $r(L_y) = h(L_y) - A h(L_x) - c$ rather than the full activation. **Right**: affine regressions explain a large fraction of downstream activation variance in Pythia-1.4B transition pairs and in Gemma-2-9B, where mean R^2 decreases as the distance between chosen layers increases.

Training SAEs independently at each layer, however, ignores a structural property of the transformer residual stream: each layer is not independent of the ones before it. Each sub-module of the architecture can be thought of as reading from some subspace of the residual stream and writing an update back into it Elhage et al. (2021), so a substantial fraction of any given layer’s activation is information carried forward or lightly transformed from earlier layers Raghu et al. (2017); Kornblith et al. (2019). Figure 1 demonstrates that simple affine regressions between selected layers explain a large fraction of downstream activation variance in both Pythia-1.4B and Gemma-2-9B. Therefore, training per-layer SAEs independently on raw activations wastes capacity relearning this predictable carried-forward structure. Moreover, multi-layer interventions become difficult to implement, since if the same information appears in multiple layerwise dictionaries, simultaneous feature replacement can produce non-additive effects as features duplicate, cancel, or drive activations out of the SAE’s training distribution across layers.

We propose *Residualized Sparse Autoencoders* (ReSAEs) to address this. For each consecutive pair of selected layers, we fit an affine map from the earlier layer’s activation to the later one, and train the later-layer SAE on the residual, which is the component not explained by this map. At reconstruction time, the SAE output is composed with the fitted affine chain to recover the original activation space, keeping ReSAEs directly comparable to standard SAEs at evaluation. This forces each SAE to encode what is new at its layer rather than what is already predictable from the layer below. Moreover, the effect of linear interventions performed at one layer can directly be viewed at the next layer through this affine transformation.

Our experiments show that residualization improves multi-layer SAE behavior in the intended direction. ReSAEs reconstruct a strictly harder target (reflected in lower raw explained variance) yet recover transformer cross entropy more faithfully under simultaneous multi-layer intervention, produce less redundant decoder directions, and outperform raw SAEs on downstream sparse probing and targeted perturbation evaluations. Our contributions are:

1. We introduce Residualized Sparse Autoencoders (ReSAEs), which train each per-layer SAE on the component of its layer’s activation not linearly predictable from the previous selected layer, and compose SAE reconstructions with a fitted affine chain to recover original activation space.
2. We show that ReSAEs recover more cross entropy under multi-layer intervention despite lower explained variance, and produce more orthogonal decoder directions (evidence that removing predictable cross-layer structure directs SAE capacity toward layer-local information that is more useful for intervention).
3. We report evaluations across reconstruction quality, cross-entropy degradation, decoder similarity, sparse probing, targeted probe perturbation, and spurious correlation removal on Pythia-1.4B and Gemma-2-9B across multiple sparsity levels.

2 Prior Work

Sparse Autoencoders for Mechanistic Interpretability. Sparse autoencoders (SAEs) have emerged as a central tool for decomposing transformer hidden states into interpretable feature directions Cunningham et al. (2023); Bricken et al. (2023). Early work trains ReLU SAEs on individual layers of small language models and shows that individual latent directions correspond to interpretable token or context properties. BatchTopK Gao et al. (2025) and JumpReLU Rajamanoharan et al. (2024) variants improve training stability and feature utilization, and are now standard for large-scale sweeps. These approaches all train on a *single* selected layer; the multi-layer setting is largely unexplored.

Multi-Layer and Hierarchical SAEs. Matryoshka SAEs Bussmann et al. (2025) train nested groups of latents across layers by sharing a prefix of latent codes, allowing coarse-to-fine decomposition. Cross-Coders Lindsey et al. (2024) jointly train on pairs of adjacent layers to identify features that are shared across layers. Both approaches use raw activations and are susceptible to the cross-layer redundancy problem that motivates this work.

Cross-Layer Redundancy and Interventions. Hidden states across transformer layers are strongly correlated Raghu et al. (2017); Kornblith et al. (2019). This redundancy has been studied for transfer learning and representation compression, but its impact on SAE intervention quality has received little attention. Elhage et al. (2021) characterize superposition and residual stream structure in transformers, motivating a separation of “new” versus “carried-forward” information. Our work operationalizes this separation via affine regression between selected layers, a tool previously used for layer similarity analysis but not for SAE training targets.

Intervention Stability. Online SAE interventions (replacing a model’s activations with SAE reconstructions during a forward pass) accumulate errors across layers because each SAE sees an out-of-distribution input caused by upstream replacement errors. Templeton et al. (2024) and subsequent work note that single-layer interventions degrade downstream computation. Multi-layer simultaneous intervention is an open problem and we use a simple interaction-residual metric, explained in Section 4.1, to measure whether single-layer intervention results predict multi-layer behavior.

3 Methodology

3.1 Problem Setup

Let $L = (\ell_1, \ell_2, \dots, \ell_M)$ be a strictly increasing subset of transformer layer indices, $\ell_1 < \ell_2 < \dots < \ell_M$. For token position i in sequence b , let $h_{b,i}^{(\ell_m)} \in \mathbb{R}^d$ denote the hidden state at selected layer ℓ_m . The goal is to train M independent sparse autoencoders, one per selected layer, such that:

1. each SAE encodes only the information *newly* introduced at its layer rather than information carried forward from earlier selected layers;
2. multi-layer online interventions compose without compounding error.

3.2 Pairwise Affine Residualization

For each consecutive pair (ℓ_m, ℓ_{m+1}) in L , we fit an affine map

$$\begin{aligned} h_{b,i}^{(\ell_{m+1})} &\approx A_m h_{b,i}^{(\ell_m)} + c_m, \\ A_m &\in \mathbb{R}^{d \times d}, \quad c_m \in \mathbb{R}^d, \end{aligned} \tag{1}$$

using ridge-regularized ordinary least squares on a held-out calibration set; implementation details, including calibration-set size, are given in Section 4. The residualized activation at layer ℓ_{m+1} is

$$r_{b,i}^{(\ell_{m+1})} = h_{b,i}^{(\ell_{m+1})} - (A_m h_{b,i}^{(\ell_m)} + c_m). \tag{2}$$

The regression chain is defined over the chosen subset L , not over all model layers. For the anchor layer ℓ_1 , we center by the empirical mean: $\tilde{h}^{(\ell_1)} = h^{(\ell_1)} - \mu_1$, where $\mu_1 = \mathbb{E}[h^{(\ell_1)}]$.

3.3 Block Normalization

Residual blocks have different norms at different depths, so we apply per-block RMS normalization:

$$S_m = \frac{1}{\sigma_m}, \quad \sigma_m = \sqrt{\mathbb{E}[\|r^{(\ell_m)}\|_2^2/d]} + \varepsilon. \tag{3}$$

The normalized input to the SAE at block m is $z^{(m)} = S_m r^{(\ell_m)}$ (with $z^{(1)} = S_1 \tilde{h}^{(\ell_1)}$ for the anchor).

3.4 Independent Per-Layer SAE Training

Each block trains its own BatchTopK SAE Gao et al. (2025) with its own encoder and decoder. For block m , the SAE model is

$$\begin{aligned} a_{b,i}^{(m)} &= W_e^{(m)} z_{b,i}^{(m)} + b_e^{(m)}, \\ f_{b,i}^{(m)} &= \text{TopK}_k \left(\text{ReLU} \left(a_{b,i}^{(m)} \right) \right), \\ \hat{z}_{b,i}^{(m)} &= W_d^{(m)} f_{b,i}^{(m)} + b_d^{(m)}, \end{aligned} \quad (4)$$

where $W_e^{(m)}, b_e^{(m)}$ are the encoder parameters, $W_d^{(m)}, b_d^{(m)}$ are the decoder parameters, and TopK_k keeps only the largest k nonnegative activations. The full latent is $f_{b,i} = (f_{b,i}^{(1)}, \dots, f_{b,i}^{(M)})$. The per-block loss is the reconstruction objective under this hard sparsity constraint:

$$\begin{aligned} \mathcal{L}_{\text{SAE}}^{(m)} &= \mathbb{E}_{b,i} \left[\frac{1}{d} \left\| z_{b,i}^{(m)} - \hat{z}_{b,i}^{(m)} \right\|_2^2 \right], \\ \left\| f_{b,i}^{(m)} \right\|_0 &\leq k. \end{aligned} \quad (5)$$

The total objective is blockwise separable:

$$\mathcal{L}_{\text{SAE}} = \sum_{m=1}^M \mathcal{L}_{\text{SAE}}^{(m)}, \quad (6)$$

where gradients from block m' do not update the SAE for block $m \neq m'$. All selected layers share a single activation-collection pass for computational efficiency, but the SAE objectives and parameters remain fully layer-local.

3.5 Original-Space Reconstruction

After encoding, we recursively reconstruct original-space activations:

$$\hat{h}_{b,i}^{(\ell_1)} = \mu_1 + S_1^{-1} \hat{z}_{b,i}^{(1)}, \quad (7)$$

$$\hat{h}_{b,i}^{(\ell_{m+1})} = A_m \hat{h}_{b,i}^{(\ell_m)} + c_m + S_{m+1}^{-1} \hat{z}_{b,i}^{(m+1)}. \quad (8)$$

Equation (8) applies for $m = 1, \dots, M-1$. This chain means each later layer depends on the recursively reconstructed earlier layer, and evaluation metrics are computed on the resulting $\hat{h}^{(\ell_m)}$ in original space.

3.6 Online Multi-Layer Intervention

During evaluation, we apply online hooks in layer order within a single forward pass. Let $\tilde{h}_{b,i}^{(\ell_m)}$ denote the activation that arrives at selected layer ℓ_m during this modified forward pass, after any earlier hooks have already changed the residual stream. Let $\hat{h}_{b,i}^{(\ell_m)}$ denote the activation written back by the hook.

For a raw SAE, the hook at layer ℓ_m encodes the current activation $\tilde{h}_{b,i}^{(\ell_m)}$ and writes the raw SAE reconstruction in original coordinates. This is simple, but later raw SAEs may receive inputs produced by earlier replacements rather than by the unmodified model, so their inputs can drift away from the clean activation distribution used for training.

For a ReSAE, the hook uses the same affine decomposition as training, but conditions the prediction term on the previous reconstructed selected layer. For $m > 1$, the residual input to the layer ℓ_m SAE is

$$u_m = S_m \left(\tilde{h}_{b,i}^{(\ell_m)} - (A_{m-1} \hat{h}_{b,i}^{(\ell_{m-1})} + c_{m-1}) \right), \quad (9)$$

and the hook writes

$$\hat{h}_{b,i}^{(\ell_m)} = A_{m-1} \hat{h}_{b,i}^{(\ell_{m-1})} + c_{m-1} + S_m^{-1} \hat{z}^{(m)}(u_m). \quad (10)$$

The anchor layer ℓ_1 is reconstructed as in Equation (7). Intuitively, ReSAE hooks subtract the part of the current activation that is predictable from the previous reconstructed selected layer before applying the SAE. Thus, if an upstream replacement mainly changes a later activation through the learned affine relationship, that propagated change is removed from the residual input. The later SAE then sees an input closer to its training target, rather than a full activation shifted by earlier hooks.

4 Experimental Setup

4.1 Evaluation Metrics

Cross-entropy and overinteraction. Let $\text{CE}(S)$ be the language-model cross entropy after replacing the activations at the layer set $S \subseteq L$ with SAE reconstructions during an online forward pass. Let $\text{CE}(\emptyset)$ denote the unmodified model. We define

$$\begin{aligned}\Delta_S &= \text{CE}(S) - \text{CE}(\emptyset), \\ \Delta_S^{\text{add}} &= \sum_{\ell \in S} \Delta_{\{\ell\}}, \\ \text{OI}(S) &= \Delta_S - \Delta_S^{\text{add}}.\end{aligned}\tag{11}$$

We introduce an additivity-residual metric, which we call overinteraction (OI), to quantify whether multi-layer replacement behaves like the sum of its single-layer effects. Specifically, we compare the observed CE degradation from jointly replacing a set of layers to the sum of CE degradations from replacing each layer independently. Values near zero indicate approximately additive composition, while deviations from zero indicate that the layers interact under joint replacement.

Reconstruction quality. We report explained variance (EV) in reconstructed activation space, where after decoding each SAE representation back into the model’s original residual-stream coordinates, EV measures how much variance in the clean activation is recovered. We also report cross-entropy degradation under two replacement modes. In teacher-forced replacement, each SAE receives the clean activation from an unmodified forward pass, isolating the quality of each reconstruction from error propagation. In sequential-online replacement, hooks are applied in layer order during a single modified forward pass, so later layers receive activations produced after earlier replacements; this measures the intervention setting that matters when multiple SAEs are used together.

Feature quality. We use feature-quality evaluations from SAE Bench Karvonen et al. (2025). Sparse probing measures whether a small number of SAE latents can linearly recover task labels on Bias-in-Bios, Amazon reviews, GitHub code, AG News, and Europarl. Targeted probe perturbation (TPP) measures whether ablating latents associated with a target concept selectively changes the corresponding classifier score, while spurious correlation removal (SCR) tests whether ablating latents associated with a spurious attribute removes that attribute without destroying task-relevant signal Karvonen et al. (2024a). We also include first-letter splitting and absorption evaluations, following the observation that SAE features can split or absorb related concepts across latents Chanin et al. (2025). For SCR and TPP, we distinguish two intervention budgets. The *global* budget ablates the top n latents across all selected-layer dictionaries, so the intervention uses n features total. The *per-layer joint* budget ablates the top n latents separately in each selected layer and applies all of those ablations in one forward pass, so for Pythia’s four selected layers it uses $4n$ total ablated features.

4.2 Models and Data

We train SAEs on two transformer language models using the monology/pile-uncopyrighted dataset. For **Pythia-1.4B-deduped** Biderman et al. (2023), we use the layer set $\{0, 6, 12, 18\}$. For **Gemma-2-9B** Gemma Team et al. (2024), a 42-layer model, we use three layer sets for the layer-spacing study. We call the common distance between consecutive selected layers the *layer gap*: gap 4 denotes $\{2, 6, 10, 14, 18, 22, 26\}$, gap 6 denotes $\{2, 8, 14, 20, 26\}$, and gap 8 denotes $\{2, 10, 18, 26\}$.

All SAEs use the BatchTopK objective with dictionary size $N = 65,536$. Pythia sweeps sparsity $k \in \{40, 80, 160, 320\}$, while Gemma uses $k = 160$ across the layer-set sweep. Each SAE is trained for 5×10^8 tokens. The affine cross-layer regressions are calibrated on 5×10^5 tokens before SAE training begins. We use Adam with learning rate 3×10^{-4} , a 1000-step warmup, and 20% learning-rate decay.

For every model and layer set, we compare two SAE families. The raw baseline trains standard BatchTopK SAEs directly on the unresidualized activations. The residualized model trains SAEs on pairwise affine residuals and reconstructs original-space activations with the regression chain described above.

5 Results

The main result is that ReSAEs recover cross entropy better under multi-layer replacement despite having lower explained variance. We present the evidence in this order. First, we show that residualization lowers raw reconstruction

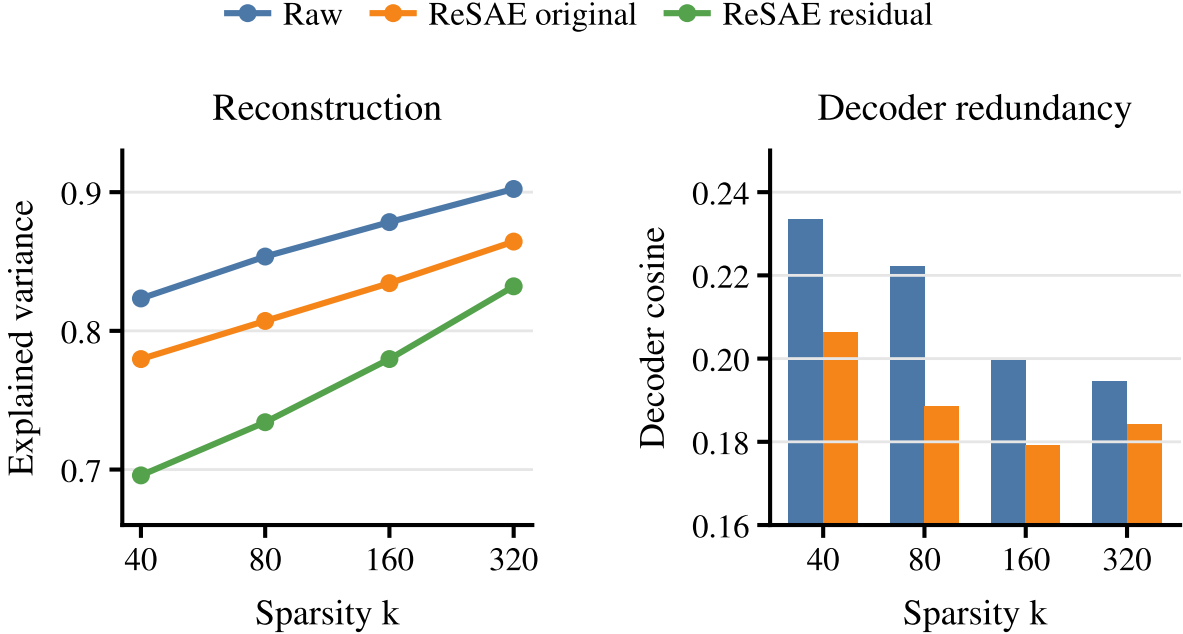


Figure 2: Pythia-1.4B reconstruction and decoder geometry. **Left:** raw original-space EV, ReSAE original-space EV, and ReSAE residual-target EV. **Right:** mean max decoder cosine similarity; lower means less redundant decoder directions.

fidelity but reduces decoder redundancy. Second, we show that this lower EV does not imply worse functional reconstruction: ReSAEs recover CE better in the multi-layer setting that motivates the method. Third, we test whether the learned latents remain useful for readout and targeted ablation. Finally, we repeat the central comparisons on Gemma-2-9B to check whether the pattern persists at larger scale and across different layer spacings.

5.1 Lower EV but Less Redundant Dictionaries

We first quantify the representation tradeoff introduced by residualization. Explained variance (EV) measures how much clean activation variance is recovered, while decoder cosine similarity measures whether different learned directions duplicate one another. Figure 2 compares these quantities for the Pythia-1.4B sparsity sweep. Raw SAEs have higher original-space EV at every sparsity level (0.823–0.902 vs. 0.780–0.864), which is expected because ReSAEs train on the harder residual target and then reconstruct through the affine chain. The residual-space EV of ReSAEs is lower still (0.696–0.832), confirming that residualization removes a substantial linearly predictable component before SAE training.

However, ReSAEs are less redundant. They have lower mean max decoder cosine similarity at every k , with the largest gap at $k = 80$ (0.189 vs. 0.222). Thus EV favors the raw baseline, but decoder geometry favors residualization: ReSAEs recover less raw activation variance, while learning a less duplicated feature basis.

5.2 Recovered CE Improves Despite Lower EV

We next evaluate whether lower EV translates into worse model behavior. Recovered cross entropy measures the task-level effect of replacing model activations with SAE reconstructions, while overinteraction measures whether multi-layer replacement behaves like the sum of the corresponding single-layer replacements. Figure 3 shows recovered cross entropy when all four selected Pythia layers are replaced. Teacher-forcing is shown for both raw SAEs and ReSAEs. The ReSAE teacher-forced curve is best at every sparsity level, reaching CE 2.398 at $k = 320$ compared with the clean-model CE of 2.260. Thus, despite lower original-space EV at every sparsity, ReSAEs recover the model’s cross-entropy behavior more faithfully when each layer is reconstructed from the clean activation. Sequential online

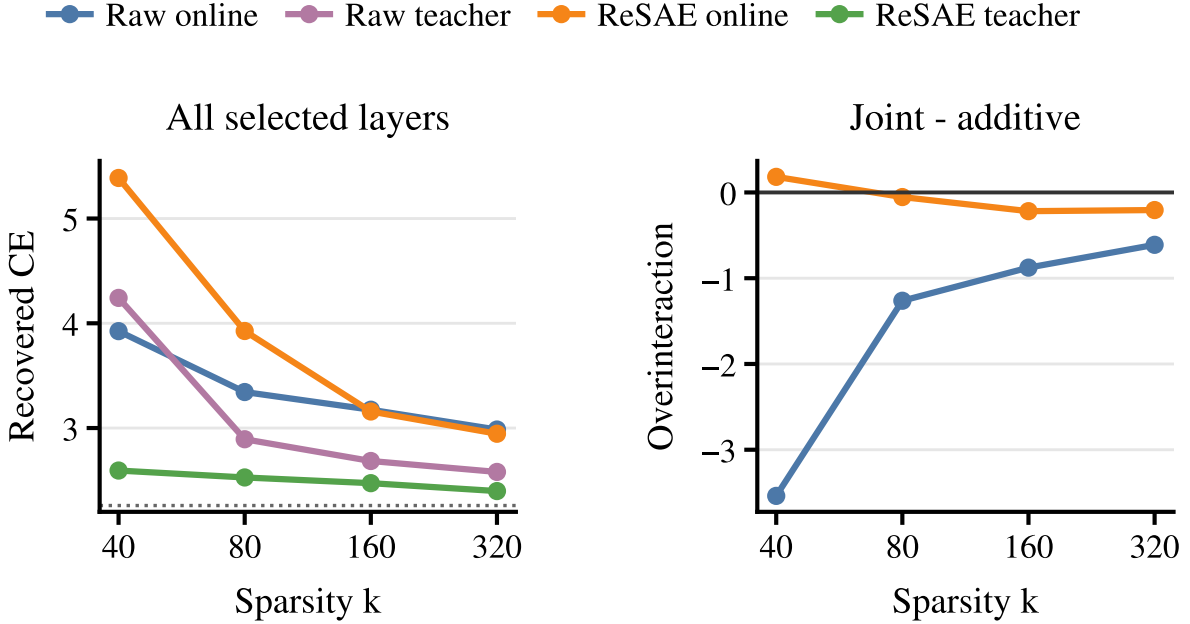


Figure 3: Pythia-1.4B recovered CE and overinteraction for all four selected layers. **Left:** recovered CE for raw and ReSAE replacement under both teacher-forced and sequential-online modes. **Right:** sequential-online overinteraction $OI(L)$; values near zero indicate more additive composition.

replacement is harder because later layers receive activations already modified by earlier replacements. In this setting, raw SAEs are better at very low sparsity, but ReSAEs overtake raw SAEs at $k = 160$ and $k = 320$.

The overinteraction results help explain this gap between EV and functional replacement quality. Raw SAEs have large negative overinteraction, especially at low k , meaning the joint intervention is far from the additive prediction obtained from single-layer replacements. ReSAEs move much closer to zero for $k \geq 80$, indicating more predictable multi-layer composition. This suggests that residualization is most useful once the sparse code has enough capacity to represent the residual target: it may reconstruct less total variance, but its layerwise replacements compose more reliably.

5.3 Feature Readout and Targeted Ablation

We then test whether residualized latents are useful beyond reconstruction. Sparse probing measures linear readout from a small number of latents, TPP measures whether ablating target-associated latents changes the intended probe score, and SCR measures whether spurious information can be removed while preserving task-relevant signal. Figure 4 summarizes sparse probing and TPP on Pythia-1.4B. ReSAEs are consistently better on sparse probing, with the largest gain at low sparsity (0.879 vs. 0.849 top-5 accuracy at $k = 40$) and near convergence by $k = 320$. On TPP with a per-layer joint budget, ReSAEs are also better through $k = 160$ and essentially tied at $k = 320$. These results suggest that residualization improves feature usefulness for linear readout and targeted perturbation, especially when the sparse budget is small.

Table 5.3 reports SCR. This stricter ablation test does not favor ReSAEs: raw SAEs usually score higher, especially under the per-layer joint budget at larger k . This indicates that residualized features are not uniformly better for every intervention objective; in this setting, joint ablations of residualized features can also remove useful causal signal.

5.4 Larger-Scale and Layer-Spacing Checks

We next test whether the same pattern holds in a larger model. The Gemma-2-9B sweep varies the spacing between selected layers while keeping sparsity fixed, testing robustness to the choice of multi-layer decomposition. Figure 5 summarizes the Gemma-2-9B layer-gap sweep. The left panel shows that ReSAEs have lower decoder cosine similarity than raw SAEs at every gap, so residualization again reduces decoder redundancy. The right panel shows cross-entropy

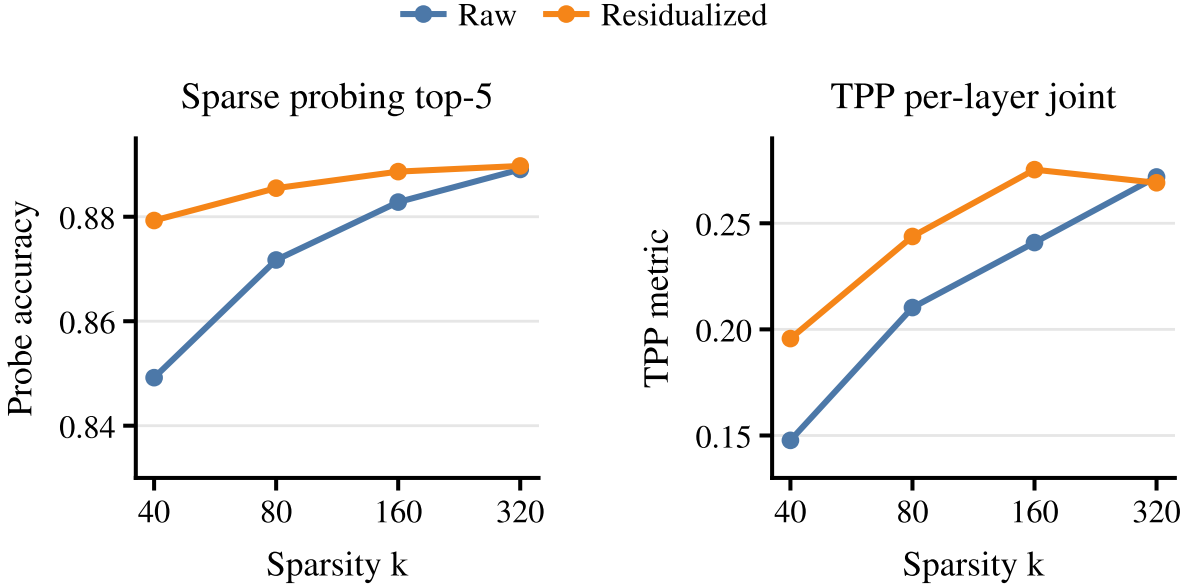


Figure 4: Pythia-1.4B feature-level results. **Left:** sparse probing accuracy using the top five SAE features across selected layers. **Right:** TPP under the per-layer joint budget. Higher is better in both panels.

k	Global Budget		Per-Layer Joint	
	Raw	ReSAE	Raw	ReSAE
40	+0.035	+0.032	+0.028	−0.012
80	+0.045	+0.002	+0.007	−0.040
160	+0.035	−0.022	−0.013	−0.124
320	+0.011	−0.007	−0.043	−0.077

Table 1: Pythia-1.4B SCR mean scores, averaged over SCR classes and ablation budgets n . Global ablates n features total; per-layer joint ablates n features per selected layer and applies the four layerwise ablations together.

degradation with separate axes for teacher-forced and sequential-online replacement. ReSAEs are better in both modes at every gap: teacher-forced ΔCE stays near zero (0.026–0.058), and sequential-online ΔCE is roughly half of the raw baseline across the sweep (0.743–1.093 vs. 1.591–2.371). Thus the central Pythia result—better recovered CE despite the reconstruction tradeoff—also appears in Gemma.

5.5 Feature-Level Results at Larger Scale

Finally, we repeat the feature-level checks on Gemma-2-9B. For the gap-4 run, Table 2 reports the results. ReSAEs improve sparse probing at top-1, top-2, and top-5. TPP is mixed under the global budget but clearly better for ReSAEs under the per-layer joint budget.

6 Discussion

6.1 Residualization trades raw EV for less redundant dictionaries

The reconstruction results show that residualization should not be judged by original-space EV alone. Raw SAEs reconstruct the full activation directly, so they naturally score higher on EV. ReSAEs instead remove the part of the

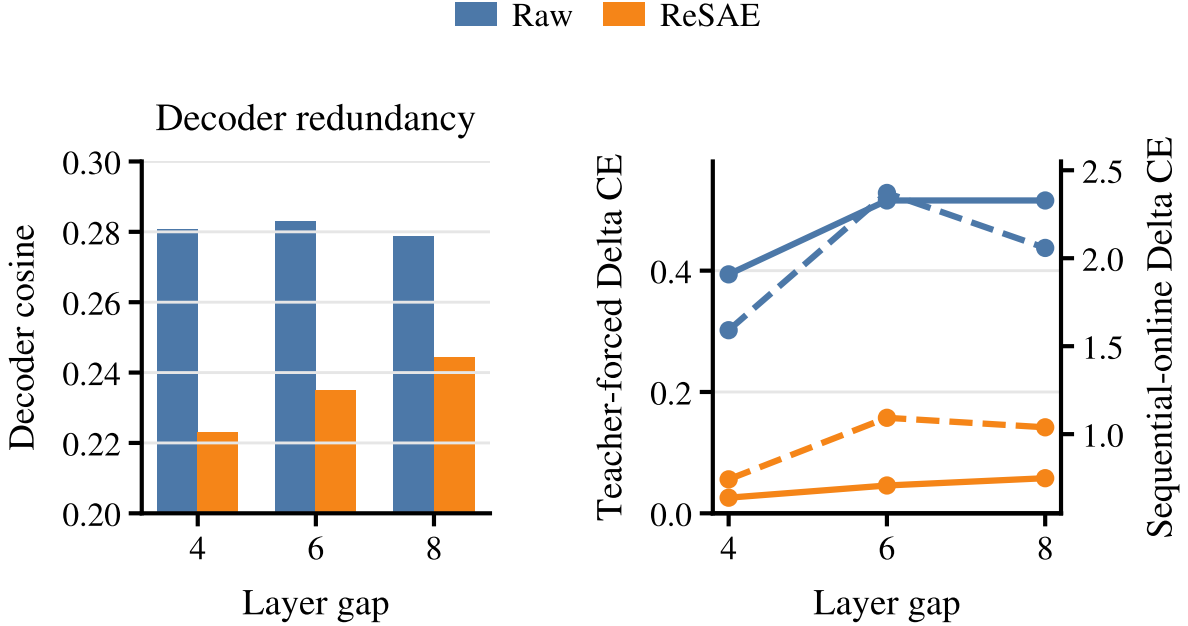


Figure 5: Gemma-2-9B layer-gap trends ($k = 160$). **Left:** mean max decoder cosine similarity by layer gap; lower indicates less redundant decoder directions. **Right:** Δ CE by layer gap. Solid lines use the teacher-forced left axis, while dashed lines use the sequential-online right axis.

Table 2: Gemma-2-9B feature-level results for the gap-4 layer set $\{2, 6, 10, 14, 18, 22, 26\}$ at $k = 160$.

Metric	Raw	ReSAE
Sparse probe top-1	0.786	0.840
Sparse probe top-2	0.817	0.859
Sparse probe top-5	0.865	0.901
TPP global budget	0.121	0.118
TPP per-layer joint	0.220	0.252

activation that is linearly predictable from an earlier selected layer and train on what remains. This residual target has less easily recoverable structure, which lowers EV, but it also reduces duplication among decoder directions.

We note that ReSAEs reconstruct less of the raw activation but recover more of the model’s cross entropy. EV measures variance recovery in a coordinate system the model itself does not privilege, whereas CE measures the functional aspect of the model. That these two quantities move in opposite directions is evidence that residualization steers capacity away from directions that carry forward predictable structure and toward directions that participate in the next layer’s computation. This is the property a sparse dictionary should have for interpretability, where features correspond to what the model does and not to what it passively transmits.

6.2 Residualization helps when the residual code is not too bottlenecked

The intervention results show that the benefit of residualization depends on whether the sparse code has enough capacity to represent the residual target. In Gemma, ReSAEs recover CE better than raw SAEs across all tested layer gaps, in both teacher-forced and sequential-online replacement. In Pythia, ReSAEs are strongest under teacher forcing and become competitive or better online once k is large enough, while low- k online replacement still favors raw SAEs. This pattern suggests a capacity cost, where if the residual code is too sparse, the affine chain cannot recover information that the SAE failed to encode. Once that bottleneck is reduced, the residual target gives the online intervention a cleaner object to replace and the layerwise replacements compose more predictably.

6.3 Feature-level gains are task-dependent

The feature-level evaluations show that residualization improves some uses of latents but not all of them. Sparse probing and TPP mostly favor ReSAEs, consistent with the view that removing carried-forward structure concentrates capacity on information introduced at the selected layer. SCR does not follow the same pattern on Pythia, where raw SAEs usually score higher, especially for per-layer joint ablations. This suggests that residualized features can be more useful for readout and targeted perturbation while still being less selective for the specific goal of removing a spurious attribute without damaging the class signal. Thus, ReSAEs improve several intervention-centered evaluations, but the best dictionary can still depend on the downstream operation.

6.4 Residualization is independent of the SAE objective

The method should be viewed as a target construction step rather than as a commitment to BatchTopK SAEs. Before training the SAE, we replace the raw activation target with the component not explained by an affine prediction from an earlier selected layer. Once that target is defined, the dictionary could be trained with other SAE objectives, including JumpReLU Rajamanoharan et al. (2024), p-Anneal Karvonen et al. (2024b), or Matryoshka-style hierarchical training Bussmann et al. (2025). Testing these combinations is an important direction for future work, since stronger SAE objectives may recover more interpretable structure from the residual target.

6.5 Affine residualization leaves open design choices

The affine map is a useful first choice because it removes a simple, measurable form of carried-forward structure. It is cheap to calibrate and keeps the residual target easy to interpret, but it is unlikely to be the only useful choice. Low-rank or block-structured maps could reduce memory cost, while nonlinear predictors might remove additional predictable structure at the risk of discarding information useful for interpretation. For example, the attention module in a transformer linearly combines embeddings at each layer, where the main non-linearity enters through the softmax similarity function. Another open question is layer selection, where the Gemma gap sweep suggests that distance between chosen layers matters, but it does not yet tell us how to choose the best layer set for a task.

7 Conclusion

ReSAEs are a small change to the SAE training target with a large effect on multi-layer behavior. Instead of having every layerwise dictionary relearn information already predictable from an earlier selected layer, they train on the residual left by an affine cross-layer model and reconstruct through that model at evaluation time. Across Pythia-1.4B and Gemma-2-9B, this reduces decoder redundancy and improves several intervention-centered metrics, including recovered CE, sparse probing, and targeted perturbation. The gains are not universal, and SCR in particular shows that residualization can lose information useful for some causal-ablation objectives. Still, the overall picture is encouraging since in multi-layer SAE interventions, treating the residual stream as a coupled object is more faithful than treating each layer as an isolated training problem.

Limitations

The experiments cover two model families and a limited set of selected layers. Pythia is evaluated with one four-layer set, while Gemma is evaluated across three regularly spaced layer sets at a single sparsity level. This is enough to show that the effect is not tied to one architecture, but not enough to claim that a particular layer gap or sparsity choice is generally optimal.

The residualization map is affine and is fit on a fixed calibration sample. This makes the method simple and easy to analyze, but it also means any nonlinear predictable structure remains in the SAE target. Conversely, a stronger predictor might remove information that is useful for interpretability. The present experiments do not settle where that boundary should be.

The evaluation suite is intervention-heavy but still incomplete. Recovered CE, overinteraction, sparse probing, TPP, and SCR each stress a different behavior, and they do not always agree. In particular, Pythia SCR favors raw SAEs in several settings. This means ReSAEs should not be treated as uniformly better dictionaries; they are better suited to some multi-layer replacement and feature-use regimes than others.

Finally, the current implementation stores full affine maps between selected layers. That is acceptable for the models studied here but could become expensive for wider models or denser layer sets. Low-rank, block-structured, or otherwise compressed regression maps are likely needed before this approach is convenient at much larger scale.

Acknowledgments

This work was supported in part by the DARPA Young Faculty Award, the National Science Foundation (NSF) under Grants #2127780, #2319198, #2321840, #2312517, and #2235472, #2431561, the Semiconductor Research Corporation (SRC), the Office of Naval Research through the Young Investigator Program Award, and Grants #N00014-21-1-2225 and #N00014-22-1-2067, Army Research Office Grant #W911NF2410360. Additionally, support was provided by the Air Force Office of Scientific Research under Award #FA9550-22-1-0253, along with generous gifts from Xilinx and Cisco.

References

- Jan Betley, Daniel Chee Hian Tan, Niels Warncke, Anna Szyber-Betley, Xuchan Bao, Martín Soto, Nathan Labenz, and Owain Evans. Emergent misalignment: Narrow finetuning can produce broadly misaligned LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 4043–4068. PMLR, 2025.
- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar Van Der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- Joseph Bloom, Curt Tigges, and David Chanin. SAELens, 2024.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nicholas L. Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Chris Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.
- Bart Bussmann, Noa Nabeshima, Adam Karvonen, and Neel Nanda. Learning multi-level features with matryoshka sparse autoencoders. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 6077–6101. PMLR, 2025.
- David Chanin, James Wilken-Smith, Tomáš Dulka, Hardik Bhatnagar, Satvik Golechha, and Joseph Isaac Bloom. A is for absorption: Studying feature splitting and absorption in sparse autoencoders. In *Advances in Neural Information Processing Systems*, 2025. NeurIPS 2025 oral.
- Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.
- Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.
- Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. In *International Conference on Learning Representations*, 2025.
- Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- Zhengfu He, Wentao Shu, Xuyang Ge, Lingjie Chen, Junxuan Wang, Yunhua Zhou, Frances Liu, Qipeng Guo, Xuanjing Huang, Zuxuan Wu, Yu-Gang Jiang, and Xipeng Qiu. Llama Scope: Extracting millions of features from Llama-3.1-8b with sparse autoencoders. *arXiv preprint arXiv:2410.20526*, 2024.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Delong Chen, Wenliang Dai, Ho Shu Chan, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- Adam Karvonen, Can Rager, Samuel Marks, and Neel Nanda. Evaluating sparse autoencoders on targeted concept erasure tasks. *arXiv preprint arXiv:2411.18895*, 2024a.
- Adam Karvonen, Benjamin Wright, Can Rager, Rico Angell, Jannik Brinkmann, Logan Smith, Claudio Mayrink Verdun, David Bau, and Samuel Marks. Measuring progress in dictionary learning for language model interpretability with board game models. In *Advances in Neural Information Processing Systems*, pages 83091–83118, 2024b.
- Adam Karvonen, Can Rager, Johnny Lin, Curt Tigges, Joseph Isaac Bloom, David Chanin, Yeu-Tong Lau, Eoin Farrell, Callum Stuart McDougall, Kola Ayonrinde, Demian Till, Matthew Wearden, Arthur Conmy, Samuel Marks, and Neel Nanda. SAEbench: A comprehensive benchmark for sparse autoencoders in language model interpretability. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 29223–29264. PMLR, 2025.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, pages 3519–3529. PMLR, 2019.

- Tom Lieberum, Senthooan Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 278–300, Miami, Florida, US, 2024. Association for Computational Linguistics.
- Jack Lindsey, Adly Templeton, Jonathan Marcus, Tom Conerly, Joshua Batson, and Chris Olah. Sparse crosscoders for cross-layer features and model diffing. *Transformer Circuits Thread*, 2024.
- Maithra Raghu, Justin Gilmer, Jason Yosinski, and Jascha Sohl-Dickstein. SVCCA: Singular vector canonical correlation analysis for deep learning dynamics and interpretability. *Advances in Neural Information Processing Systems*, 30, 2017.
- Senthooan Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. Jumping ahead: Improving reconstruction fidelity with JumpReLU sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024.
- Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R. Bowman, Newton Cheng, Esin Durmus, Zac Hatfield-Dodds, Scott R. Johnston, Shauna Kravec, Timothy Maxwell, Sam McCandlish, Kamal Ndousse, Oliver Rausch, Nicholas Schiefer, Da Yan, Miranda Zhang, and Ethan Perez. Towards understanding sycophancy in language models. *arXiv preprint arXiv:2310.13548*, 2023.
- Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L. Turner, Callum McDougall, Monte MacDiarmid, Alex Tamkin, Esin Durmus, Tristan Hume, Francesco Mosconi, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024.
- Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.